

PYMOFL: A Composable Optimization Benchmark Library for Python

Travis Silvers

firestrand@users.noreply.github.com

<https://github.com/firestrand/pyMOFL>

Abstract

We present PYMOFL, an open-source Python library for constructing and evaluating optimization benchmark functions through functional composition. Unlike existing benchmark libraries that rely on class inheritance hierarchies to combine transformations with base functions, PYMOFL decouples transformations from objective functions entirely: shifts, rotations, scaling, bias, and noise are pure callable objects composed around base functions via a generic pipeline. A declarative JSON configuration layer drives construction of arbitrarily complex benchmarks—including all 25 functions of the CEC 2005 special session—without writing function-specific code. The library provides 47 benchmark function implementations, 14 transformation types, weighted composition and hybrid function abstractions, and a modular factory system that maps configuration trees to executable function objects. All operations are vectorized through NUMPY, supporting efficient batch evaluation. PYMOFL is available under the MIT license at <https://github.com/firestrand/pyMOFL> and via PyPI.

Keywords: optimization benchmarks, metaheuristics, benchmark functions, CEC 2005, function composition, Python

1 Introduction

Benchmark functions are essential infrastructure for optimization research. They provide controlled, reproducible landscapes on which algorithms can be compared with respect to convergence speed, solution quality, scalability, and robustness to problem characteristics such as multimodality, non-separability, ill-conditioning, and noise [Jamil and Yang, 2013]. Standardized benchmark suites, most notably the series organized by the Congress on Evolutionary Computation (CEC) [Suganthan et al., 2005], have become the *de facto* protocol for evaluating metaheuristic optimizers.

Despite decades of use, the software implementations of these benchmarks remain fragmented. Researchers frequently re-implement functions from scratch, introducing subtle discrepancies in transformation order, rotation convention, or bias handling that compromise reproducibility. Existing libraries such as Opfunu [Van Thieu, 2024], COCO/BBOB [Hansen et al., 2021], and Benchmarks.jl [JuliaOpt Contributors, 2023] vary in the degree to which they expose the internal composition of transformed functions, making it difficult to extend suites or verify intermediate transformation steps.

PYMOFL addresses these challenges through three design principles:

1. **Composition over inheritance.** Transformations are pure callable objects—not subclasses of the objective function base class—composed via a generic pipeline (`ComposedFunction`).
2. **Declarative configuration.** A JSON configuration tree is the single source of truth for any benchmark variant. The nesting structure directly encodes the transformation application order.

Table 1: Comparison of optimization benchmark libraries.

	Language	Functions	CEC Suites	Composable	Config-driven
Opfunu	Python	500+	2005–2022	No	No
COCO/BBOB	C/Py	24/suite	BBOB	No	No
SciPy	Python	~15	—	No	No
Benchmarks.jl	Julia	~30	—	No	No
PYMOFL	Python	47	2005	Yes	Yes

3. **Vectorized evaluation.** All transforms and functions support batch evaluation through NUMPY array operations, enabling efficient use in population-based algorithms.

The remainder of this paper is organized as follows. Section 2 surveys related work. Section 3 describes the library architecture. Section 4 catalogs the implemented benchmark functions. Section 5 details the transformation pipeline. Section 6 explains weighted composition and hybrid function abstractions. Section 7 presents the config-driven factory system. Section 8 demonstrates CEC 2005 suite coverage. Section 9 provides usage examples. Section 10 discusses testing and quality assurance. Section 12 concludes.

2 Related Work

Several software packages provide optimization benchmark functions for research. We briefly compare the most prominent.

Opfunu [Van Thieu, 2024] offers over 500 functions spanning CEC 2005–2022 and traditional benchmarks. It uses a class-inheritance model in which each CEC function is a standalone class with hard-coded transformations. This simplifies lookup but couples transformation logic to specific functions, making it difficult to recombine transforms independently.

COCO/BBOB [Hansen et al., 2021] provides a carefully designed testbed for black-box optimization with automated performance assessment. COCO is implemented in C with Python bindings, emphasizing runtime performance but offering limited user-level extensibility for custom function composition.

SciPy optimize [Virtanen et al., 2020] includes a small collection of test functions but is not designed as a comprehensive benchmark library.

Benchmarks.jl [JuliaOpt Contributors, 2023] targets the Julia ecosystem with a collection of classical test functions. Like Opfunu, transformations are embedded within individual function implementations.

Table 1 summarizes key differences. PYMOFL’s distinguishing contribution is the separation of concerns between base functions, transformations, and composition logic, enabled by a config-driven factory pipeline.

3 Architecture

PYMOFL is organized around four layers, shown in Figure 1: base functions, transformations, compositions, and a factory system that assembles them from configuration.

3.1 Design Principles

Composition over inheritance. Optimization benchmark suites typically require the same base function (e.g., Sphere) to appear in multiple variants: shifted, rotated, shifted *and* rotated, with noise, with bias, etc. An inheritance-based design leads to a combinatorial explosion of subclasses. PYMOFL instead treats transformations as independent, composable units:

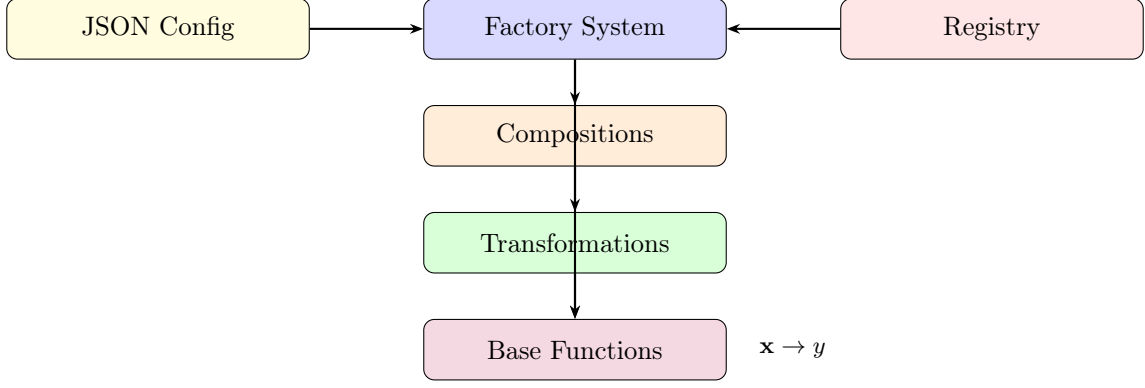


Figure 1: High-level architecture. The factory system reads JSON configuration and the function registry to assemble compositions of transformations and base functions. Evaluation flows bottom-up: input transforms are applied to $\mathbf{x}$, the base function is evaluated, and output transforms produce the final scalar y .

$$f_{\text{composed}}(\mathbf{x}) = T_{\text{out}} \circ f_{\text{base}} \circ T_{\text{in}}(\mathbf{x}) \quad (1)$$

where T_{in} and T_{out} are ordered sequences of vector and scalar transforms, respectively.

Bounds as metadata. Unlike libraries that enforce box constraints within the function evaluation, PYMOFL treats bounds as pure metadata (`Bounds` frozen dataclass with `low`, `high`, `mode`, `qtype`, `step`). Enforcement or quantization is opt-in via the `Quantized` wrapper. This prevents the benchmark implementation from interfering with the optimizer’s constraint-handling strategy.

Single source of truth. Each benchmark suite is defined by a JSON configuration file. The CEC 2005 suite, for example, is encoded in a single `cec2005_suite.json` file that specifies all 25 functions across all supported dimensions (2, 10, 30, 50). Adding a new suite (e.g., BBOB) requires only a new JSON file and any associated data files—no code changes.

4 Benchmark Functions

PYMOFL provides 47 benchmark function classes spanning classical unimodal, multimodal, and engineering design problems. Each class inherits from `OptimizationFunction` and implements the abstract `evaluate(x)` method. Table 2 summarizes the catalog.

All functions are registered with the global `FunctionRegistry` at import time via a `@register` decorator and automatic submodule scanning (`scan_package`). Each function is accessible by one or more string aliases (e.g., `"sphere"`, `"Sphere"`), enabling symbolic lookup from configuration files.

The base class provides:

- `evaluate(x: ndarray) -> float`: single-point evaluation with input validation.
- `evaluate_batch(X: ndarray) -> ndarray`: batch evaluation (default loops; overridden with vectorized implementations in concrete classes).
- `__call__`: delegates to `evaluate` with validation.
- Metadata properties: `dimension`, `initialization_bounds`, `operational_bounds`.

Table 2: Benchmark functions implemented in PYMOFL, grouped by category.

Category	Count	Functions
Unimodal	11	Sphere, Rosenbrock, Schwefel 1.2, Schwefel 2.6, Schwefel 2.13, High-Conditioned Elliptic, Dixon–Price, Powell (3 variants), Sum of Different Powers
Multimodal	27	Ackley (4 variants), Rastrigin, Griewank, Weierstrass, Katsuura, Levy, Zakharov, Alpine (2), Schaffer (6 variants), Schwefel 2.4/2.20–2.26/2.36, Lunacek Bi-Rastrigin, Easom, Goldstein–Price, McCormick, Matyas, HappyCat, HGBat
Composed	2	Griewank-of-Rosenbrock (F8F2), Expanded Schaffer F6
Engineering	4	Compression Spring, Gear Train, Network, Lennard-Jones
Utility	3	Step, Tripod, Max Absolute, Different Powers, Perm, Himmelblau, Branin (2), Bent Cigar, Discus

Table 3: Transformation types in PYMOFL.

Type	Class	Category	Mathematical Operation
Shift	ShiftTransform	Vector	$\mathbf{x}' = \mathbf{x} - \mathbf{o}$
Rotate	RotateTransform	Vector	$\mathbf{x}' = \mathbf{M}^\top \mathbf{x}$
Scale	ScaleTransform	Vector	$\mathbf{x}' = \mathbf{x}/\lambda$
Offset	OffsetTransform	Vector	$\mathbf{x}' = \mathbf{x} + \mathbf{v}$
Non-continuous	NonContinuousTransform	Vector	$x'_i = \lfloor 2x_i + 0.5 \rfloor / 2$
Indexed shift	IndexedShiftTransform	Vector	Shift on selected dimensions
Indexed scale	IndexedScaleTransform	Vector	Scale on selected dimensions
Indexed rotate	IndexedRotateTransform	Vector	Rotate on selected dimensions
Bias	BiasTransform	Scalar	$y' = y + b$
Noise	NoiseTransform	Scalar	$y' = y \cdot (1 + \epsilon)$
Normalize	NormalizeTransform	Scalar	$y' = y \cdot C / f_{\max}$
Quantize	Quantized	Special	Bounds enforcement + discretization

5 Transformation Pipeline

Transformations are the core mechanism for constructing benchmark variants. PYMOFL defines two abstract base classes for transforms:

- **VectorTransform**: $\mathbb{R}^d \rightarrow \mathbb{R}^d$ (input transforms)
- **ScalarTransform**: $\mathbb{R} \rightarrow \mathbb{R}$ (output transforms)

Both are pure callables—they carry parameters but are not `OptimizationFunction` subclasses. Table 3 lists all implemented transforms.

5.1 ComposedFunction

`ComposedFunction` is the primary composition mechanism. It is itself an `OptimizationFunction`, wrapping a base function with ordered sequences of input and output transforms:

```

1 class ComposedFunction(OptimizationFunction):
2     def evaluate(self, x):
3         z = x
4         for t in self.input_transforms:
5             z = t(z) # VectorTransform chain
6         y = self.base_function.evaluate(z)

```

```

7         for t in self.output_transforms:
8             y = t(y)                                     # ScalarTransform chain
9         return y

```

Listing 1: ComposedFunction evaluation flow.

The batch evaluation path applies each transform to the full matrix before passing to the next stage, preserving NUMPY vectorization:

$$\mathbf{Y} = T_{\text{out}}^{(m)} \circ \dots \circ T_{\text{out}}^{(1)} \circ f_{\text{base}} \circ T_{\text{in}}^{(k)} \circ \dots \circ T_{\text{in}}^{(1)}(\mathbf{X}) \quad (2)$$

where $\mathbf{X} \in \mathbb{R}^{n \times d}$ is a batch of n input vectors.

6 Composition and Hybrid Functions

Beyond simple transform chains, PYMOFL supports two higher-order composition patterns required by CEC 2005 functions F15–F25.

6.1 Weighted Composition

WeightedComposition combines K component functions using distance-based Gaussian weights:

$$f(\mathbf{x}) = \sum_{i=1}^K w_i(\mathbf{x}) \cdot \left[C_i \cdot \frac{f_i(\mathbf{x})}{f_{\text{max},i}} + b_i \right] + b_{\text{global}} \quad (3)$$

where the weight for component i is:

$$w_i(\mathbf{x}) = \frac{\hat{w}_i}{\sum_{j=1}^K \hat{w}_j}, \quad \hat{w}_i = \exp\left(-\frac{\|\mathbf{x} - \mathbf{o}_i\|^2}{2\sigma_i^2 d}\right) \quad (4)$$

Here $\mathbf{o}_i$ is the optimum of component i , σ_i controls the basin width, C_i is a normalization constant, $f_{\text{max},i}$ is the maximum function value used for normalization (pre-computed at a canonical point), and b_i is a per-component bias. Optional dominance suppression amplifies the weight of the nearest component, creating sharper transitions between basins.

Each component is a fully self-contained **ComposedFunction** with its own transform chain. The raw input $\mathbf{x}$ (without composition-level transforms) is used for distance-based weighting, ensuring weight computation reflects the true distance in the original search space.

6.2 Hybrid Function

HybridFunction partitions the input vector $\mathbf{x} \in \mathbb{R}^d$ into K disjoint subvectors $\mathbf{x}_{[s_i:e_i]}$, each evaluated by a different component:

$$f(\mathbf{x}) = \sum_{i=1}^K \alpha_i \cdot f_i(\mathbf{x}_{[s_i:e_i]}) \quad (5)$$

where α_i are optional weights and (s_i, e_i) define the partition boundaries.

7 Config-Driven Factory System

A central design goal of PYMOFL is that benchmark construction should be data-driven. The factory system translates JSON configuration trees into executable function objects through five specialized modules.

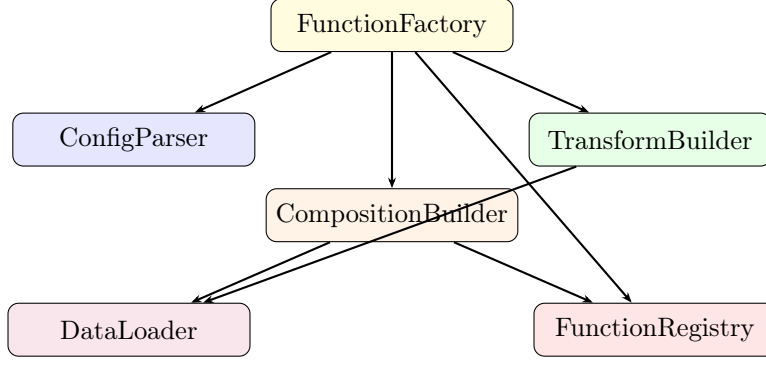


Figure 2: Factory module decomposition. **FunctionFactory** orchestrates parsing, transform construction, composition building, and registry lookup.

7.1 Configuration Structure

The JSON configuration uses a nested structure in which the outermost wrapper in the tree corresponds to the last transformation applied during evaluation:

```

1 {
2   "type": "bias",
3   "parameters": {"value": -450.0},
4   "function": {
5     "type": "rotate",
6     "parameters": {"matrix": "f03/matrix_rotation_D{dim}.txt"},
7     "function": {
8       "type": "shift",
9       "parameters": {"vector": "f03/vector_shift_D{dim}.txt"},
10      "function": {
11        "type": "high_conditioned_elliptic",
12        "parameters": {"dimension": 10}
13      }
14    }
15  }
16 }

```

Listing 2: JSON configuration for CEC 2005 F03 (Shifted Rotated High-Conditioned Elliptic).

This nesting directly encodes the evaluation order: **shift** is applied first to **x**, then **rotate**, then the base function is evaluated, then **bias** is added to the result. The **{dim}** placeholder is resolved at construction time based on the requested dimension.

7.2 Factory Modules

The factory is decomposed into five focused modules (Figure 2):

DataLoader Loads shift vectors and rotation matrices from text files, supporting dimension-templated filenames.

ConfigParser Recursively parses the nested JSON tree, collecting transforms in application order and extracting the base function specification.

TransformBuilder Instantiates **VectorTransform** and **ScalarTransform** objects from (type, parameters) tuples, loading data files as needed.

CompositionBuilder Assembles weighted compositions with per-component transform chains, optimum loading, and $f_{\max}$ pre-computation.

FunctionFactory Thin orchestrator that delegates to the above modules and returns a **ComposedFunction** or **WeightedComposition**.

Table 4: CEC 2005 benchmark functions implemented via config-driven construction.

F#	Name	Category	Transforms
1	Shifted Sphere	Unimodal	Shift, Bias
2	Shifted Schwefel 1.2	Unimodal	Shift, Bias
3	Shifted Rotated Elliptic	Unimodal	Shift, Rotate, Bias
4	Shifted Schwefel 1.2 + Noise	Unimodal	Shift, Noise, Bias
5	Schwefel 2.6 (Bounds)	Unimodal	Special
6	Shifted Rosenbrock	Multimodal	Shift, Bias
7	Shifted Rotated Griewank	Multimodal	Shift, Rotate, Bias
8	Shifted Rotated Ackley (Bounds)	Multimodal	Shift, Rotate, Bias
9	Shifted Rastrigin	Multimodal	Shift, Bias
10	Shifted Rotated Rastrigin	Multimodal	Shift, Rotate, Bias
11	Shifted Rotated Weierstrass	Multimodal	Shift, Rotate, Bias
12	Schwefel 2.13	Multimodal	Special
13	Expanded Griewank+Rosenbrock	Expanded	Shift, Bias
14	Shifted Rotated Schaffer F6	Expanded	Shift, Rotate, Bias
15	Hybrid Composition 1	Composition	10 components
16	Rotated HC 1	Composition	10 comp. + rotate
17	Rotated HC 1 + Noise	Composition	10 comp. + noise
18	Rotated HC 2	Composition	10 components
19	Rotated HC 2 Narrow Basin	Composition	10 components
20	Rotated HC 2 (Bounds)	Composition	10 components
21	Rotated HC 3	Composition	10 components
22	Rotated HC 3 High Condition	Composition	10 components
23	Non-Continuous Rotated HC	Composition	10 comp. + NC
24	Rotated HC 4	Composition	10 components
25	Rotated HC 4 (No Bounds)	Composition	10 components

8 CEC 2005 Suite Coverage

PYMOFL provides complete coverage of the CEC 2005 special session on real-parameter optimization [Suganthan et al., 2005], all 25 functions across dimensions 2, 10, 30, and 50. The suite is defined entirely in `cec2005_suite.json` (1,204 lines), with associated shift vectors and rotation matrices stored as text files organized by function number.

Table 4 catalogs the CEC 2005 functions by category.

The composition functions (F15–F25) are the most structurally complex, combining 10 component functions—each with per-component shift, scale, rotation, and normalization—into a weighted composition with Gaussian distance-based weighting and optional dominance suppression.

8.1 Extensibility to Other Suites

Because suite construction is entirely config-driven, adding support for additional benchmark suites (e.g., CEC 2013, CEC 2017, BBOB) requires:

1. A JSON configuration file defining the function tree for each benchmark.
2. Any associated data files (shift vectors, rotation matrices).
3. Implementation of any base functions not already in the registry.

No factory code, composition code, or transformation code needs to change.

9 Usage Examples

9.1 Direct Function Evaluation

```

1 import numpy as np
2 from pyMOFL.functions.benchmark import SphereFunction, AckleyFunction
3
4 sphere = SphereFunction(dimension=10)
5 x = np.zeros(10)
6 print(sphere.evaluate(x)) # 0.0
7
8 ackley = AckleyFunction(dimension=10)
9 print(ackley.evaluate(x)) # 0.0 (global minimum)

```

Listing 3: Basic function evaluation.

9.2 Explicit Composition

```

1 from pyMOFL.functions.benchmark import SphereFunction
2 from pyMOFL.functions.transformations import (
3     ComposedFunction, ShiftTransform, RotateTransform, BiasTransform,
4 )
5
6 base = SphereFunction(dimension=2)
7 shift = ShiftTransform(np.array([1.0, 2.0]))
8 rotation = RotateTransform(np.eye(2)) # identity rotation
9 bias = BiasTransform(-450.0)
10
11 composed = ComposedFunction(
12     base_function=base,
13     input_transforms=[shift, rotation],
14     output_transforms=[bias],
15 )
16 # Optimum at [1.0, 2.0] with value -450.0
17 print(composed.evaluate(np.array([1.0, 2.0])))

```

Listing 4: Building a transformed function programmatically.

9.3 Config-Driven Construction

```

1 import json
2 from pathlib import Path
3 from pyMOFL.factories import DataLoader, FunctionFactory,
4     FunctionRegistry
5
6 base = Path("src/pyMOFL/constants/cec/2005")
7 with open(base / "cec2005_suite.json") as f:
8     suite = json.load(f)
9
10 loader = DataLoader(base_path=base)
11 factory = FunctionFactory(data_loader=loader, registry=
12     FunctionRegistry())
13
14 # Build F01 (Shifted Sphere) in 10 dimensions
15 f01_config = suite["functions"]["F01"]["dimensions"]["10"]
16 f01 = factory.create_function(f01_config)
17
18 x_opt = np.array(...) # loaded shift vector
19 print(f01.evaluate(x_opt)) # -450.0

```

Listing 5: Building CEC 2005 F01 from the suite configuration.

9.4 Batch Evaluation

```
1 # Evaluate 1000 random points in one call
2 X = np.random.uniform(-100, 100, size=(1000, 10))
3 results = f01.evaluate_batch(X) # shape: (1000,)
```

Listing 6: Vectorized batch evaluation for population-based algorithms.

10 Quality Assurance

PYMOFL maintains a comprehensive test suite of 908 tests organized to mirror the source tree structure. Testing follows a test-driven development (TDD) methodology.

10.1 Test Categories

- **Unit tests** for each benchmark function: known optima, dimensionality validation, batch consistency.
- **Transformation tests**: correctness of each transform type, batch vs. single-point equivalence, composition order verification.
- **Factory tests**: config parsing, transform building, end-to-end construction for all CEC 2005 functions.
- **CEC 2005 validation tests**: suite parity checks ensuring factory-constructed functions produce expected values at known optima.
- **Composition tests**: Gaussian weighting correctness, dominance suppression behavior, hybrid partitioning.

10.2 Code Quality Tools

- **pytest**: test runner with 908 passing tests.
- **ruff**: linting and formatting (single tool, zero warnings).
- **ty**: type checking (zero diagnostics on source tree).
- **PEP 561**: `py.typed` marker for downstream type checking.

11 Availability and Installation

License: MIT

Repository: <https://github.com/firestrand/pyMOFL>

PyPI: `pip install pyMOFL`

Python: ≥ 3.12

Dependencies: `NUMPY` ≥ 1.26 , `matplotlib` ≥ 3.8 (core); `typer`, `rich` (optional CLI)

Build system: `hatchling`; managed with `uv`

12 Conclusion

PYMOFL demonstrates that optimization benchmark libraries benefit from the same separation-of-concerns principles that guide application software design. By decoupling base functions from transformations and driving construction from declarative configuration, the library achieves

three practical outcomes: (1) new benchmark variants can be defined without writing code, (2) transformation logic is tested and verified independently of specific functions, and (3) the entire CEC 2005 suite is reproduced from a single JSON file plus data assets.

Future work includes support for additional CEC competition years (2013, 2017, 2022), the BBOB/COCO suite, constrained optimization benchmarks, and performance profiling infrastructure for comparing algorithm implementations across the function catalog.

Acknowledgments

The CEC 2005 benchmark definitions follow the technical report by Suganthan et al. [2005]. Rotation matrices and shift vectors are distributed with the library under the terms of the original competition materials.

References

- Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tušar, and Dimo Brockhoff. COCO: A platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software*, 36(1):114–144, 2021. doi: 10.1080/10556788.2020.1808977.
- Momin Jamil and Xin-She Yang. A literature survey of benchmark functions for global optimization problems. *International Journal of Mathematical Modelling and Numerical Optimisation*, 4(2):150–194, 2013. doi: 10.1504/IJMMNO.2013.055204.
- JuliaOpt Contributors. Benchmarks.jl: A collection of benchmark functions for optimization, 2023. URL <https://github.com/JuliaOpt/Benchmarks.jl>.
- P. N. Suganthan, N. Hansen, J. J. Liang, K. Deb, Y.-P. Chen, A. Auger, and S. Tiwari. Problem definitions and evaluation criteria for the CEC 2005 special session on real-parameter optimization. Technical Report KanGAL Report 2005005, Nanyang Technological University, May 2005.
- Nguyen Van Thieu. Opfunu: An open-source Python library for optimization benchmark functions. *Journal of Open Research Software*, 12(1):8, 2024. doi: 10.5334/jors.508.
- Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C. J. Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.